

CS208 Lab Assignment 3: Closest Pair of Points

Deadline: June.3/4/5. Late submission is not allowed

Verification Method: In-person demonstration during lab sessions.

Requirements

Task 1: Divide and Conquer Algorithm Implementation

Implement the **closest pair of points** problem using a **divide-and-conquer algorithm** with $O(n \log n)$ time complexity. You may use **C++** or **Java**.

Problem Description

Given n points in the 2-D plane (x_i, y_i) , find the **minimum Euclidean distance** between any two distinct points. If multiple pairs achieve the same minimum, returning any one is acceptable.

Your program must:

- Read points from a text file or console (format specified below).
- Output the **minimum distance** (floating point, with sufficient precision).
- Implement the divide-and-conquer algorithm that runs in $O(n \log n)$ time.

Input Format

- First line: integer n (number of points)
- Next n lines: two floats x, y separated by whitespace.

Output Format

Your program should output a single floating-point number: the minimum distance. For verification, you may also log the pair(s), but the automated script will compare distance values.

Sample Input & Output

Sample 1

Input:

```
4
0.0 0.0
2.0 3.0
5.0 1.0
1.0 2.0
```

Output (minimum distance between (2.0,3.0) and (1.0,2.0) is $\sqrt{2}$):

```
1.414213562
```

Sample 2

Input:

```
6
0 0
2 0
4 0
6 0
3 1e-9
3 -1e-9
```

Output (minimum distance between (3,1e-9) and (3,-1e-9)):

```
0.000000002
```

Note: The output should be a floating-point number. You can use a tolerance of 10^{-9} when comparing against the brute-force result.

Implementation Notes

1. Divide-and-conquer:

- Pre-sort points by x-coordinate once ($O(n \log n)$).
- Recursively split into left/right halves.
- Merge using the strip of width 2δ , with points in the strip sorted by y-coordinate (use merge sort to maintain $O(n \log n)$ overall).
- Ensure the algorithm runs in $O(n \log n)$.

2. **Correctness guarantee:** The distance computed by your divide-and-conquer algorithm must equal the true minimum distance.
3. **Efficiency constraints:**
 - For $n = 100,000$, C++: ≤ 1 second, Java: ≤ 2 seconds.
 - Memory usage: $O(n)$.

Task 2: Construct Test Cases

Design **test cases** covering the following **7 categories**.

Category	Purpose
Random points	Validate general correctness on random distribution. Randomly generated coordinates should cover all four quadrants.
Collinear points	All points lie on a horizontal line ($y = \text{constant}$).
Vertical split	Many points share nearly the same x-coordinate, testing split robustness.
Duplicate points	Some points have identical coordinates $\rightarrow$ minimum distance = 0.
Large coordinates	Coordinates up to 10^9 to test numerical stability (use double).
Worst-case strip pattern	Points arranged so the strip contains many candidates.
Performance	Generate random point sets with $n = 2^{10}, 2^{12}, 2^{14}, 2^{15}, 2^{16}, 2^{17}, 2^{18}$ to test efficiency.

- You may write **scripts** to generate these test cases automatically (e.g., Python or bash).
- **At least 10 test cases** covering the **1-6 categories**
- **At least 7 test cases** covering the **7th categories- Performance testing**
- All generated files must be placed in the `testcases/` folder.

Task 3: Test Your Algorithm

In this task you will verify the **correctness** and **performance** of your divide-and-conquer implementation.

3.1 Brute-Force Reference

- Implement a separate **brute-force program** $O(n^2)$ that computes the exact minimum distance by checking all pairs.

- This program will be used **only for correctness testing on small inputs** (e.g., $n \leq 1000$).

3.2 Correctness Verification

- **Comparison:** For each test case ($n \leq 1000$), compare the output of your divide-and-conquer algorithm with the brute-force result.
- **Automation:** Use a script (`verify.sh` / `verify.bat` / `verify.py`) that:
 - Compiles both programs.
 - Runs both solvers on every test case.
 - Reports `[PASS]` if distances match (absolute difference $< 10^{-9}$), otherwise `[FAIL]` .

3.3 Performance Testing

- **Objective:** Verify that the divide-and-conquer algorithm runs in $O(n \log n)$ time.
- **Steps:**
 - Baseline benchmark:** Measure the running time for $n = 10^5$ random points and ensure it finishes within the time limits(C++ ≤ 1 second, Java: ≤ 2 seconds).
 - Scalability analysis:** Measure execution times for the following n values:
 $n = 2^{10}, 2^{12}, 2^{14}, 2^{15}, 2^{16}, 2^{17}, 2^{18}$ (or up to your machine limit). Plot time vs. $n \log_2 n$ to confirm $O(n \log n)$ complexity(similar to the Stable Matching assignment).
 - Analysis:** Conclude whether your algorithm's growth follows $n \log n$. Report any deviations and explain possible reasons (caching, recursion overhead, etc.).

Task 4: Automated Verification & Performance Script

Required Project Structure

```

closestpair/
|
├─ src/                # Source code
|
├─ bin/                # Compiled executables
|   ├─ bruteforce      # Brute-force executable
|   └─ closestpair     # D&C executable
|
├─ testcases/          # Input point files (generated by script)
├─ output/             # Results, logs
├─ time_analysis/      # CSV data and plot(optional)
├─ README.md           # Project introduction: compile/run steps, description of test ca
└─ verify.sh / verify.bat / verify.py    # Automated verification script

```

Automated Verification & Performance Script

You must provide a **single runnable script** named `verify.sh` (Linux/macOS) or `verify.bat` (Windows) or `verify.py` (cross-platform). The script must perform the following steps automatically:

1. **Clean** – Remove previous binaries, test outputs, and time logs.
2. **Compile** – Build the brute-force and divide-and-conquer executables. Put them in a `bin/` directory.
3. **Generate test data** – Create all required test case types and store them in `testcases/`.
4. **Correctness verification (brute-force vs D&C)** – For all correctness test cases ($n \leq 1000$), run both algorithms and compare distances. Output a clear `[PASS]` or `[FAIL]` for each test case.
5. **Time measurement** – Run the divide-and-conquer algorithm on a set of increasing sizes (e.g., $n = 2^{10}$ to 2^{18} or up to your machine limit) and produce a CSV file `time_data.csv` with columns: `n`, `nlogn`, `time_avg_ms` in `time_analysis/`. Optionally also plot the curve using a script (`gnuplot` , `matplotlib` , etc.).
6. **Summary** – Print a summary of all passed/failed correctness tests and indicate where the time data was saved.

Sample script output:

===== Step 1: Clean =====

Cleaned bin/, output/, testcases/ and time_analysis/

===== Step 2: Compile =====

Compilation successful.

===== Step 3: Generate test cases =====

Generated 10 correctness test cases in testcases/

Generated 7 performance test cases in testcases/

===== Step 4: Correctness verification =====

[PASS] random_100.txt

[PASS] random_1000.txt

[PASS] collinear_200.txt

[PASS] vertical_split_500.txt

[PASS] vertical_split_1000.txt

[PASS] duplicates_500.txt

[PASS] large_coords_600.txt

[PASS] large_coords_1000.txt

[PASS] worstcase_strip_800.txt

[PASS] worstcase_strip_1000.txt

===== Step 5: Time measurement =====

Running n from 1024 to 262144...

Time data saved to time_analysis/time_data.csv

Plot saved to time_analysis/time_plot.png(optional)

===== Summary =====

All correctness tests passed. See time_analysis/time_data.csv for n log n verification.

In-Person Verification & Demonstration Workflow

Step	Action	Expected Outcome
1	Check directory structure	Matches required layout exactly
2	Clean environment test	Delete bin/ , output/ , testcases/ and time_analysis/ ; then run verify.sh / verify.bat / verify.py

Step	Action	Expected Outcome
3	Script execution	Script runs without errors, generates test data, passes all correctness tests, produces time data and plot(optional)
4	Output inspection	bin/ contains executables, testcases/ contains test data; output/ contains correctness logs; time_analysis/ contains CSV
5	Manual check	TAs may require testing a specific test case to see if the answer meets expectations.
6	Conceptual Q&A	Answer 1–2 questions (e.g., why strip check needs y-sorting, why ≤ 7 points, how you handle duplicates, why algorithm is $O(n \log n)$)
7	README review	Compile/run steps, description of test case categories and purposes, time experiment setup.

Grading Rubric & Deduction Policy (Total: 100 points)

Category	Points	Deduction Rules
Project Structure	0	-5 if directory layout mismatch requirements; 0 if correct.
Automated Verification Script	10	-10 if script fails to compile, generate test cases, run verification, or produce time data; -5 if not fully automated (requires manual steps).
Correctness (Brute vs D&C)	60	-60 if failed the specific test case, -10 per failed categorie, capped at -60 (6 self-designed categories). Any mismatch (distance difference $> 1e-9$) $\rightarrow$ 0 for that case.
Performance Testing	20	-20 if Benchmark test failed. -10 if Scalability test does not meet expectations.
Documentation (README.md)	10	Missing or lacks: compile/run steps, description of test case types, time experiment setup $\rightarrow$ -5 to -10 .
Conceptual Q&A	0	-5 to -10 if unable to explain y-sorting, why ≤ 7 points, or time complexity reasoning. 0 if correct.

- **Plagiarism or AI-generated code without attribution:** 0 (subject to academic integrity policy)